Floating point Arithmetic

[64-bit]

A Computer Systems Project

Report Submitted for the

Master of Technology in

Computer Science & Engineering

By

SHUBHAM THAWAIT
VAIBHAV BARSAIYAN
PRASHANT YADAV



INDIAN INSTITUTE OF TECHNOLOGY
ROPAR (140001)

Under the Guidance –

Prof. Neeraj Goel Sir

DECLARATION BY THE CANDIDATES

We the undersigned solemnly declare that the report of the project work entitled **"Floating point Arithmetic 64-bit"** is based on our own work carried out during the courseof our study under the supervision of **Prof. Neeraj Goel.** We assert that the statements made and conclusions drawn are an outcome of the project work.

SHUBHAM THAWAIT
VAIBHAV BARSAIYAN
PRASHANT YADAV

ACKNOWLEDGEMENT

Every work is source which requires support from many people and areas. It gives us proud privilege to complete the project on **"Floating point Arithmetic 64-bit"** under valuable guidance and encouragement of our guide **Prof. NEERAJ GOEL** for providing all facilities and every help for smooth progress of project. We would also like to thank all the staff member of Computer Science & Engineering Department and Library of Indian Institution of Technology Ropar for timely help and inspiration for completion of the project.

VAIBHAV BARSAIYAN
PRASHANT YADAV
SHUBHAM THAWAIT

INTRODUCTION

In the landscape of numerical computing, precision and accuracy are paramount. The IEEE 754 standard has been a cornerstone in defining the rules for floating-point arithmetic, and the 64-bit format, in particular, has garnered immense significance.

Performing IEEE 64-bit floating-point arithmetic calculations involves adhering to the standards outlined in the IEEE 754 specification. The 64-bit format, also known as double precision, allocates bits for the sign, exponent, and significand (or mantissa). Here's a step-by-step guide to performing basic arithmetic calculations in IEEE 64-bit format:

Addition:

*Align Exponents:

- Identify the number with the smaller exponent.
    - Shift the significand (and adjust the exponent accordingly) of the smaller exponent until both exponents match.

*Add Significands:

- Add the significands of the aligned numbers.
- Normalize the result if necessary (shift the binary point left and increment the exponent).

. Check for Overflow:

- Ensure that the result does not exceed the maximum representable value.

. Round and Normalize:

- Round the result if needed and normalize the final result.

Subtraction:

. Align Exponents:

- Similar to addition, align the exponents by shifting the smaller exponent's significand.

. Subtract Significands:

- Subtract the significands of the aligned numbers.

- Normalize the result if necessary.

. Check for Underflow:

- Ensure that the result is within the representable range.

. Round and Normalize:

- Round and normalize the final result.

Multiplication:

. Multiply Significands:

- Multiply the significands of the two numbers.
- Add the exponents and adjust the result.

. Check for Overflow:

- Ensure that the result does not exceed the maximum representable value.

. Round and Normalize:

- Round the result if needed and normalize.

Special Cases:

. Zero and Infinity:

- Handle cases where the result is zero or infinity.

. NaN (Not a Number):

- Define behavior for operations involving NaN.

Software Requirement and Specification

Device Requirements:

PCs, Laptops

Frontend languages:

HTML, CSS

Backend language-

Javascript

Vanilla JS -->
"Vanilla JS" refers to the use of plain, standard JavaScript without the inclusion of any additional libraries or frameworks. In other words, it is the use of JavaScript in its raw, native form as supported by web browsers. The term "vanilla" is used metaphorically to convey the idea of the plain, default version of JavaScript without any added flavors or enhancements.

When developers refer to using Vanilla JS, they are typically emphasizing the use of core JavaScript functionality and features provided by the browser's scripting engine, without relying on external libraries like jQuery or frameworks like React, Angular, or Vue.js.

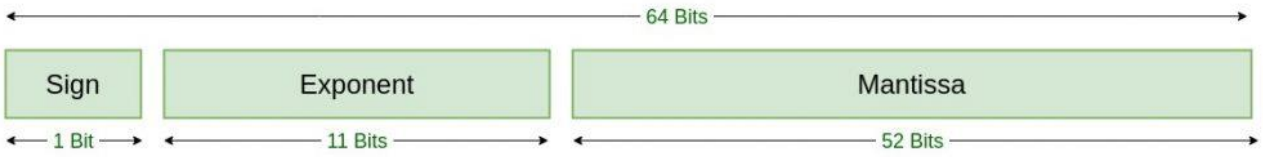
Working Principle

IEEE 64-bit floating point representation

Sign Bit	Biased Exponent	Trailling Significand bit or Mantissa

1bit 11bits 52bits

Here we use $2^{11-1} - 1 = 1023$ as bias value.



Double Precision

EXAMPLE OF DECIMAL TO IEEE FLOATING POINT CONVERSION

$(45.45)_{10}=(101101.011100)_2$

$101101.011100 = 1.01101011100 \times 2^5$

Here bias exponent = 5 + 1023=1028= $(10000000100)_2$

mantissa=01101011100

0	10000000100	01101011100110011001100……

1-bit 11 -bits 52- bits

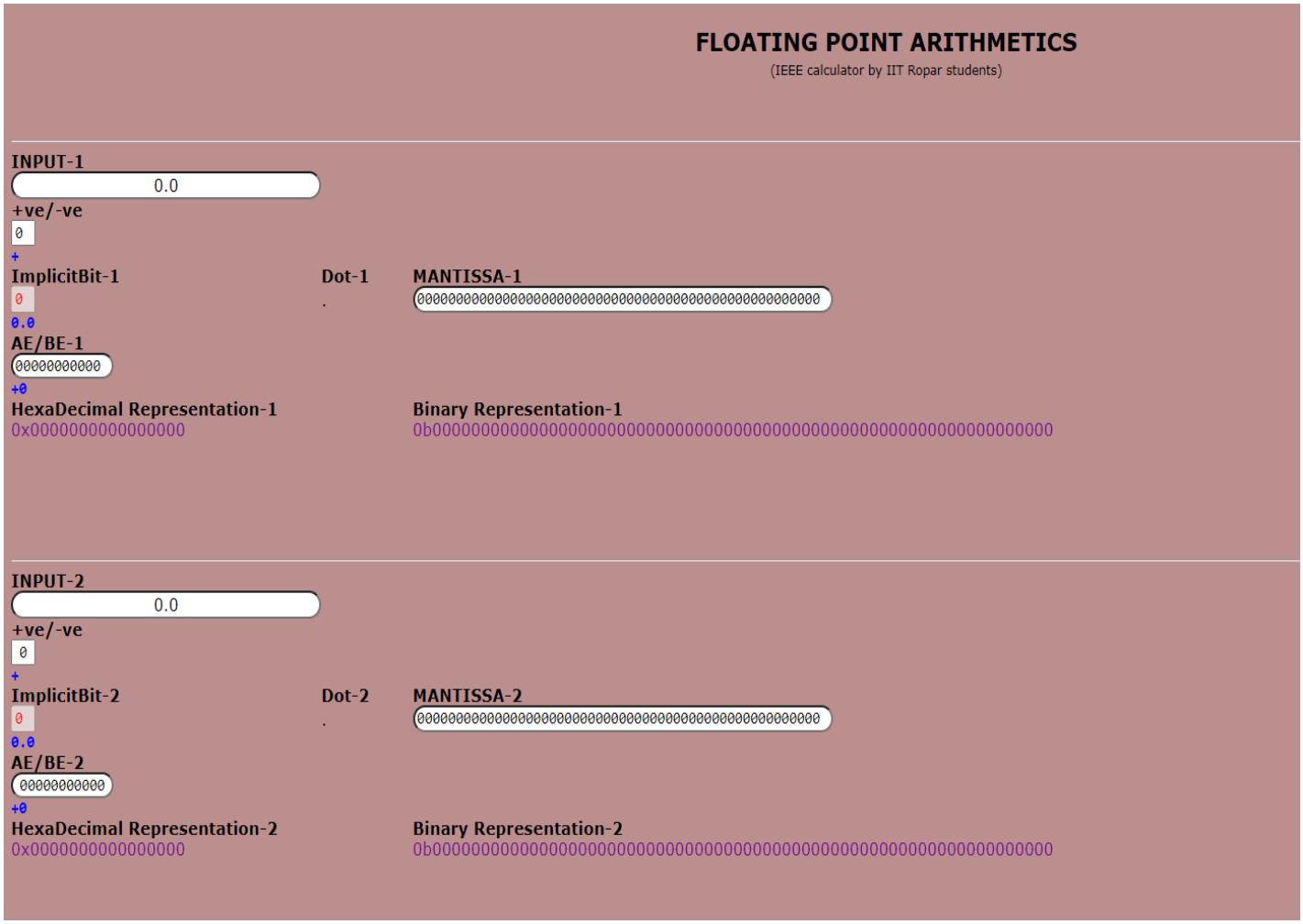
Process =>

Step1-First we give the input in decimal numbers and our website convert the decimal number into IEEE Floating point representation on the basis of above principle.

Step2-Select the operation that you want to perform.

Step3-display the final result along with its IEEE Floating point representation.

FEATURES OF WEBSITE



1) Sign(+ve/-ve) --> Our code represents the sign of the input, sign 0 means positive
Sign 1 means negative

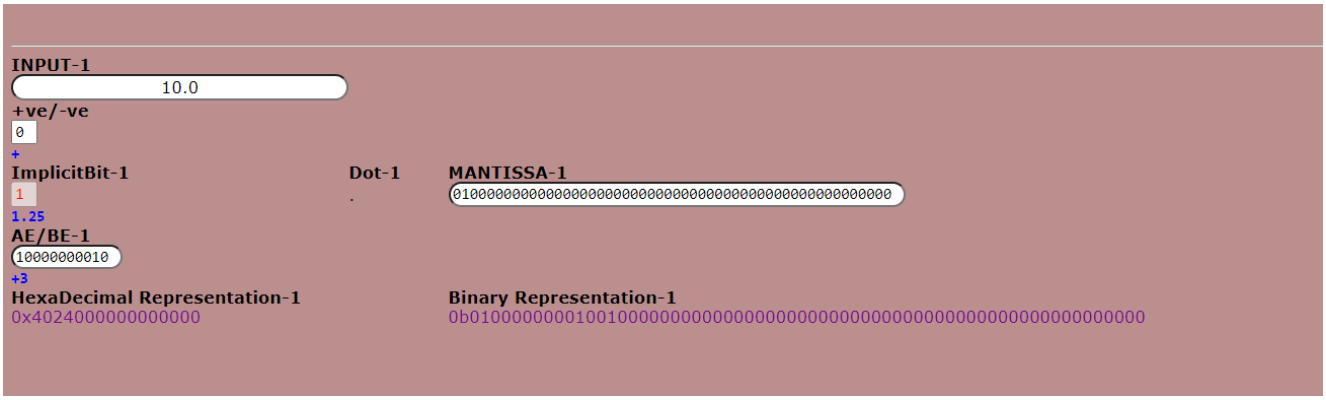
2) Implicit form --> We are doing conversion in implicit form.

3) Mantissa and Exponent (AE/BE) --> we are representing both of it in our website.

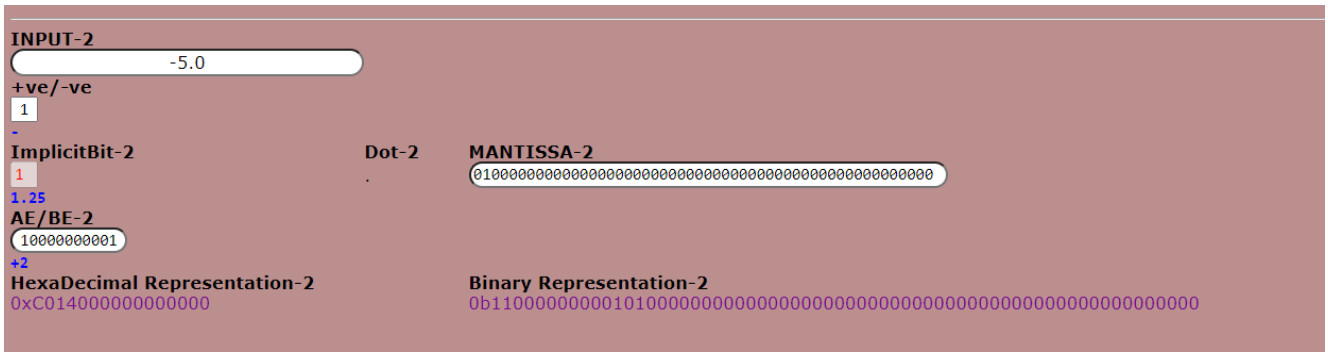
4) We are representing the Mantissa into hexadecimal representation binary representation.

OPERATIONS

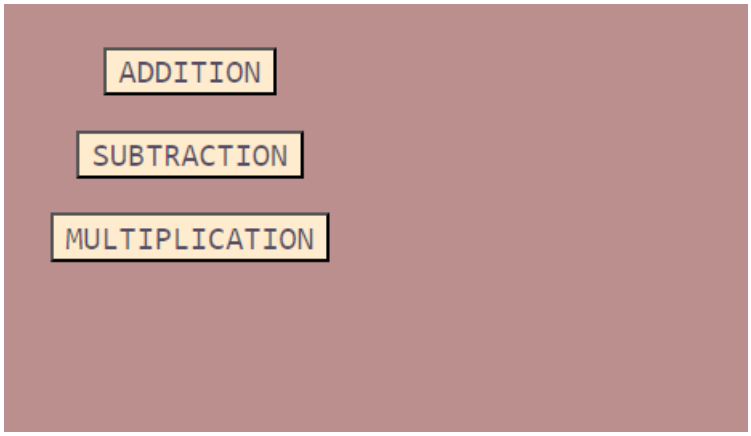
TASK1=>Taking input1



TASK 2=> Taking input 2.

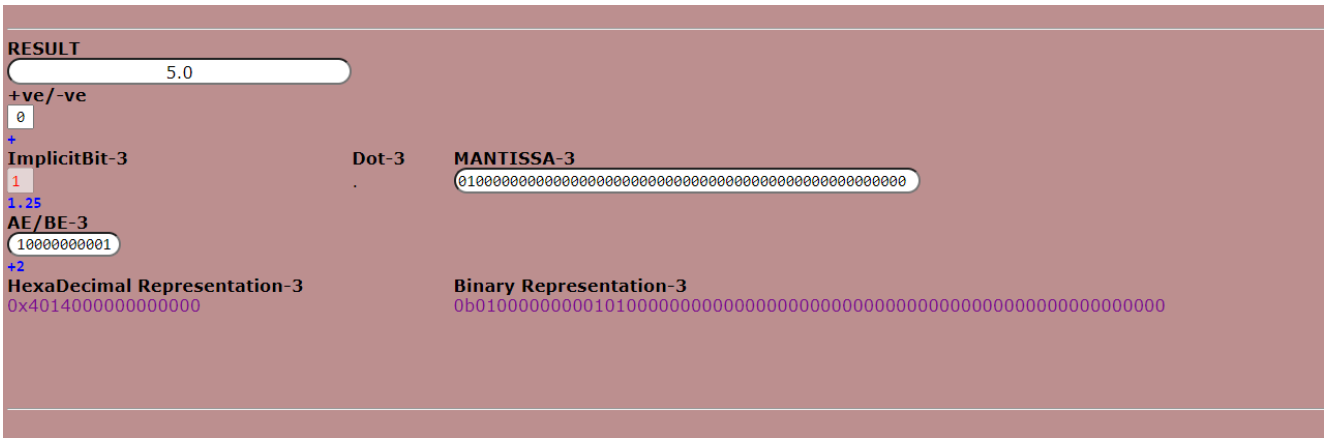


TASK 3=> SELECT THE OPERATION THAT YOU WANT TO PERFORM.

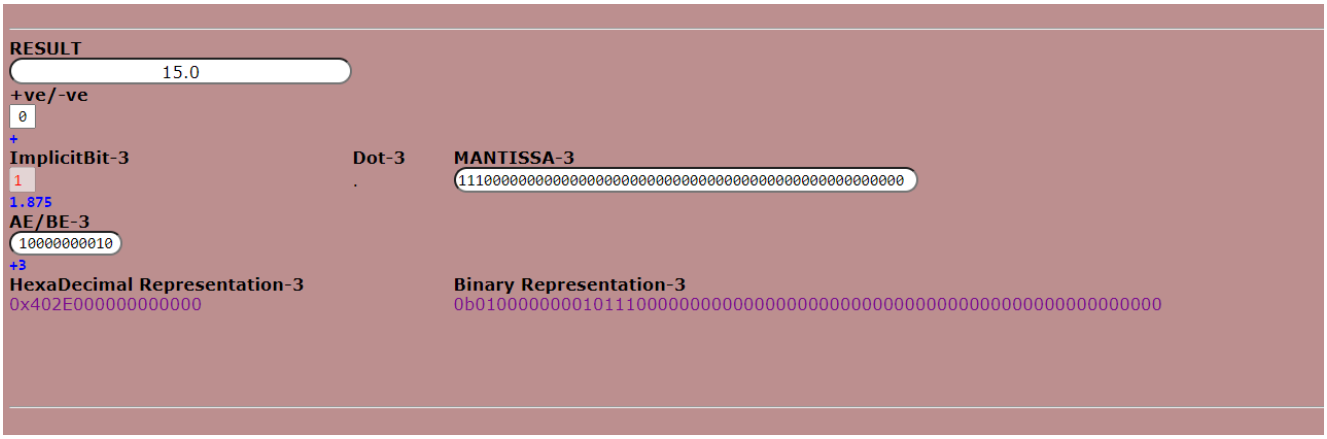


RESULT AND PERFORMANCE ANALYSIS

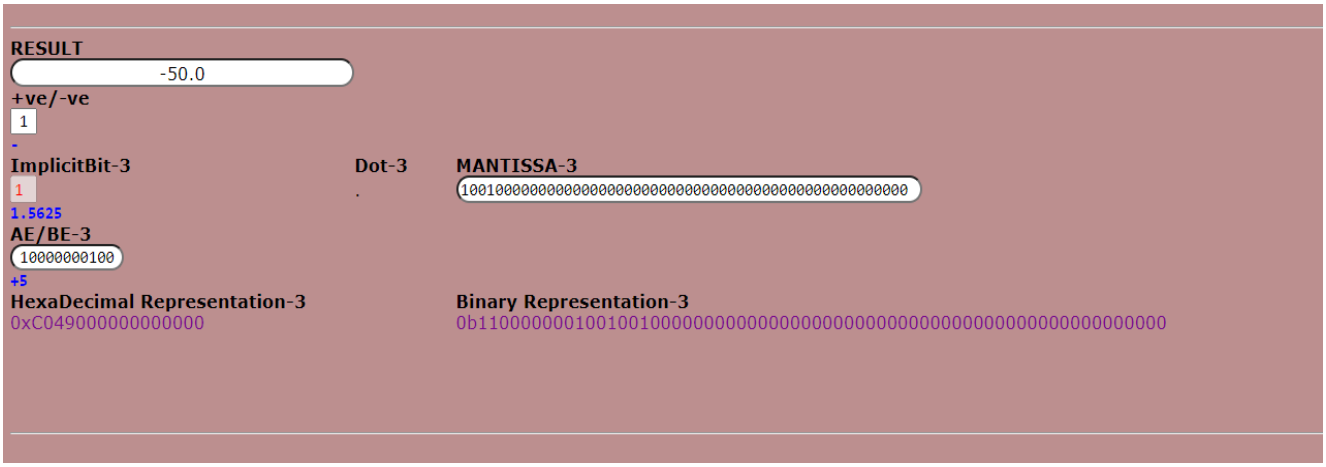
I) RESULT AFTER ADDITION



II) RESULT AFTER SUBTRACTION



III) RESULT AFTER MULTIPLICATION



ADVANTAGES AND DISADVANTAGES

ADVANTAGES -->

Increased Precision:- The 64-bit format provides a larger number of bits for the significand (mantissa) compared to 32-bit floating-point arithmetic. This increased precision allows for more accurate representation of real numbers, reducing rounding errors in complex calculations.

Extended Range:-With a larger exponent range in 64-bit floating-point numbers, the format can represent a broader range of values, both very small and very large. This is crucial in scientific and engineering applications where a wide dynamic range of values may be encountered.

Improved Accuracy in Scientific Computations:-Scientific simulations and calculations often involve very small or very large numbers. The 64-bit format minimizes the risk of overflow or underflow in such scenarios, providing a more accurate representation of physical phenomena.

Enhanced Numerical Stability:-Numerical stability is crucial in iterative algorithms and simulations. The higher precision of 64-bit arithmetic can mitigate issues related to accumulated rounding errors, resulting in more stable and reliable numerical computations.

Better Representation of Decimal Numbers:-In financial calculations and applications where exact representation of decimal values is important, the 64-bit format provides better accuracy compared to 32-bit, as it can represent more significant digits.

Standardization and Portability:-The IEEE 754 standard ensures consistency in the representation of floating-point numbers across different platforms and programming languages. This standardization facilitates portability of code and results, allowing for more reliable cross-system compatibility.

Compatibility with Modern Hardware:-Modern processors are optimized for 64-bit operations. Utilizing 64-bit floating-point arithmetic aligns well with the capabilities of contemporary hardware, potentially leading to improved computational efficiency.

Reduced Quantization Effects:-In applications where small changes in values are crucial (e.g., graphics rendering or scientific simulations), the finer precision offered by 64-bit arithmetic can reduce quantization effects, leading to smoother and more accurate results.

DISADVANTAGES-->

Complex Debugging :- Debugging numerical issues in 64-bit arithmetic can be more complex than in 32-bit, especially when dealing with subtle precision-related bugs. Identifying and resolving such issues may require more sophisticated debugging techniques.

Limited Precision in Extreme Cases :- While 64-bit floating-point arithmetic provides significantly increased precision, there are situations in scientific computing where even higher precision (e.g., 128-bit or more) may be necessary to avoid numerical instability or inaccuracies in extremely sensitive calculations.

CHALLENGES AND LIMITATIONS-->

1] Limited Precision:
The 64-bit format has a limited precision, which means that not all real numbers can be represented exactly. Some numbers may be subject to rounding errors, leading to inaccuracies in calculations.

2]Finite Representation of Real Numbers:
The set of real numbers is infinite, but the representation in a 64-bit format is finite. This leads to the inability to represent certain real numbers exactly, resulting in rounding errors.

3]Loss of Significance:
When subtracting two nearly equal numbers, there can be a significant loss of precision, known as catastrophic cancellation. This occurs when the difference between two numbers is much smaller than the numbers themselves, leading to a loss of significant digits.

4 Overflow and Underflow:
The finite range of representable numbers in 64-bit format means that calculations may result in overflow (numbers too large to be represented) or underflow (numbers too close to zero). Overflow and underflow can lead to infinities or denormalized numbers, impacting the accuracy of computations.

5 Denormalized Numbers:
Very small numbers close to zero are represented using denormalized numbers. While this extends the range of representable values, it comes at the cost of reduced precision.

CONCLUSION AND FUTURE WORK

CONCLUSION

Project Overview:

The IEEE Floating-Point Calculator is a web-based tool that allows users to interact with IEEE floating-point numbers. The calculator supports binary and decimal representations, enabling users to perform arithmetic operations on IEEE floating-point numbers.

Key Features:

1. Input Handling:
   - Users can input IEEE floating-point numbers in binary or decimal form.
   - Input is received through HTML input elements and processed as strings.

2. Arithmetic Operations:
   - The calculator supports addition, subtraction, multiplication, and division of IEEE floating-point numbers.
   -Special cases like NaN, infinity, and denormalized numbers are appropriately handled.

3. Interactive Interface:
   - The web interface dynamically updates as users input numbers and perform operations.
   - Visual representation of binary and hexadecimal forms of the IEEE numbers is provided.

4. Dynamic Bit Size:

-Users can choose between different bit sizes (16,32,64,or 128) to view and perform calculations on IEEE numbers.

5. Error Handling:
   - The calculator includes error handling for invalid inputs and NaN results.

Code Overview:

1. Input Processing:

- Binary and decimal inputs are parsed and converted into the internal representation of IEEE floating-point numbers.

  - Input validation ensures correct formatting.

2. Arithmetic Operations:

  - Functions for addition, subtraction, multiplication, and division are implemented.

  - Special cases, such as NaN and infinity, are handled appropriately.

3. Dynamic Interface:

  - The web interface is dynamically updated using JavaScript and HTML.

  - Global constants are adjusted based on the selected bit size.

4. Callback Functions:

  -Callback functions are used to update the interface and perform calculations in response to user actions.

Conclusion:

The IEEE Floating-Point Calculator provides a user-friendly platform for understanding and working with IEEE floating-point numbers. Its dynamic interface, support for different bit sizes, and accurate handling of special cases make it a valuable tool for both learning and practical use. The code's structure and comments enhance readability and maintainability.

Future Enhancements in short:

1. Enhanced User Guidance:

  - Provide tooltips or help sections for users to better understand each input field and button.

2. Optimization:

  - Optimize code for performance, especially for larger bit sizes.

3. Additional Arithmetic Functions:

  - Expand functionality to include additional arithmetic functions or advanced operations.

4. Responsive Design:

- Ensure a responsive design for better user experience on different devices.

Acknowledgments:

The project leverages JavaScript for dynamic interactions and HTML for the user interface. Special thanks to any external libraries or frameworks used in the development.

FURTHER WORK

1**. Graphical Representation:**
Enhance the visual representation of IEEE floating-point numbers by incorporating graphical elements, such as charts or graphs. This can help users better understand the distribution of numbers and the impact of different components (sign, exponent, mantissa).

2**. Custom Bit Sizes:**
Allow users to input custom bit sizes for IEEE floating-point numbers, providing flexibility beyond the predefined options. This feature could cater to specific needs or educational scenarios.

3. **Real-Time Collaboration**:
Implement real-time collaboration features, enabling multiple users to interact with and observe calculations simultaneously. This can be useful for educational purposes or collaborative problem-solving.

4**. Floating-Point Standards**:
Extend the calculator to support other floating-point standards beyond IEEE 754, such as IBM's hexadecimal floating-point format or alternative binary floating-point standards.

5. **Advanced Operations**:
Introduce support for more advanced mathematical operations, such as square root, logarithm, and trigonometric functions. This expansion can enhance the calculator's utility for scientific and engineering applications.

6**. Error Analysis**:
Implement a feature that analyzes and explains errors in calculations, helping users understand the limitations of floating-point arithmetic and potential precision issues.

7. **Unit Testing**:
Develop a comprehensive unit testing suite to ensure the accuracy and reliability of the calculator, especially when dealing with edge cases and different bit sizes.

8. **Accessibility and Internationalization:**
Improve accessibility features to ensure the calculator is usable by individuals with disabilities. Additionally, consider internationalization support for users in different regions.

9. **Offline Mode**:
Enable offline functionality, allowing users to use the calculator without an internet connection. This can be beneficial in educational environments or areas with limited connectivity.

10**. Integration with Educational Platforms:**
Explore integration possibilities with educational platforms or learning management systems. This can facilitate the use of the calculator in academic settings.

11. **User Preferences:**
Implement user preference settings, allowing users to customize the calculator interface based on their preferences, such as theme selection or default bit size.

REFERENCES

1. **IEEE 754 Standard**:
   - Title: IEEE Standard for Floating-Point Arithmetic (IEEE 754)
   - Link: [IEEE 754 Standard] (https://ieeexplore.ieee.org/document/8766229)

2. **Floating-Point Arithmetic**:
   - Book: "Computer Organization and Design: The Hardware/Software Interface" by David A. atterson and John L. Hennessy.
   -Link:[ComputerOrganization & Design]
   (https://www.elsevier.com/books/computer-organization-and design/patterson/978-0-12-407726-3)

3. **JavaScript Documentation**:
   - MDN Web Docs: JavaScript
   -Link:[MDNWebDocs](https://developer.mozilla.org/en-S/docs/Web/JavaScript)

4. **Fraction.js Library**:
   - Library for working with fractions in JavaScript.
   - Link: [Fraction.js](https://github.com/infusion/Fraction.js)

5. **Math.js Library**:
   - An extensive mathematics library for JavaScript.
   - Link: [Math.js](https://mathjs.org/)

6. **Web Development**:
   - Book: "Eloquent JavaScript" by Marijn Haverbeke.
   - Link: [Eloquent JavaScript](https://eloquentjavascript.net/)

7. **HTML and CSS**:

- MDN Web Docs: HTML
- MDN Web Docs: CSS
- Links: [HTML](https://developer.mozilla.org/en-US/docs/Web/HTML)
, [CSS](https://developer.mozilla.org/en-US/docs/Web/CSS)

8. **Web Accessibility**:
   - Web Content Accessibility Guidelines (WCAG) by W3C.
   - Link: [WCAG](https://www.w3.org/WAI/WCAG21/quickref/)

9. **Testing**:
   - Jest: JavaScript Testing Framework.
   - Link: [Jest](https://jestjs.io/)

10. **Vanilla Js** :
   -A light weight js framework for smaller projest
   -http://vanilla-js.com/

13. **Documentation**:
   - Documenting APIs: A guide to writing documentation for APIs.
   - Link: [Documenting APIs](https://idratherbewriting.com/learnapidoc/)